

## B Additional Quantitative Results

In this section, we provide additional quantitative experiment results. In Section B.1, we study how varying the number of composed trajectories  $K$  affects the performance of our method. In Section B.2, we investigate the effect of inverse dynamic models of different horizons. Next, we provide a sampling time comparison of Decision Diffuser and the proposed parallel and autoregressive sampling schemes in Section B.3. Following this, in Section B.4, we analyze how the proposed autoregressive sampling scheme performs when using different denoising starting directions.

### B.1 Number of Composed Trajectories

In Table 8, we compare CompDiffuser with GSC over composing different numbers of trajectories (results are also shown in Figure 7). Our method performs steadily when composing different numbers of trajectories while GSC collapses.

| # Comp       | PointMaze Giant Stitch |                   |                   |                   |                   |                   |
|--------------|------------------------|-------------------|-------------------|-------------------|-------------------|-------------------|
|              | 7                      | 8                 | 9                 | 10                | 11                | 12                |
| GSC          | <b>21</b> $\pm$ 1      | <b>21</b> $\pm$ 3 | <b>15</b> $\pm$ 2 | <b>2</b> $\pm$ 1  | <b>0</b> $\pm$ 0  | <b>0</b> $\pm$ 0  |
| CompDiffuser | <b>51</b> $\pm$ 7      | <b>53</b> $\pm$ 6 | <b>55</b> $\pm$ 4 | <b>56</b> $\pm$ 2 | <b>50</b> $\pm$ 6 | <b>50</b> $\pm$ 3 |

Table 8: **Quantitative Results over Different Numbers of Composed Trajectories.** We report success rates (w/o replanning) of composing 7 to 12 trajectories in the OGBench PointMaze Giant Stitch dataset. CompDiffuser can consistently construct feasible trajectories over various numbers of composed trajectories while GSC gradually collapses.

### B.2 Inverse Dynamics Model

In this section, we evaluate our planner with inverse dynamics models of different horizons (results are also shown in Figure 7). Specifically, we use an MLP to implement the inverse dynamics model, which takes as input the start and goal state and outputs an action. We use the same training dataset (as to train the planner) to train the corresponding inverse dynamics model. As shown in Table 9, our method performs steadily across different inverse dynamics model horizons, showing that the subgoals generated by our planners are of high feasibility and are robust to various inverse dynamics models’ configurations.

| Env     | Type   | Size  | 8          | 12                | 16                | 20         | 24         | 28         |
|---------|--------|-------|------------|-------------------|-------------------|------------|------------|------------|
| AntMaze | Stitch | Large | 77 $\pm$ 4 | <b>86</b> $\pm$ 2 | 80 $\pm$ 2        | 85 $\pm$ 3 | 76 $\pm$ 2 | 77 $\pm$ 3 |
|         |        | Giant | 61 $\pm$ 5 | 65 $\pm$ 3        | <b>68</b> $\pm$ 4 | 63 $\pm$ 3 | 60 $\pm$ 2 | 63 $\pm$ 3 |

Table 9: **Quantitative Results of CompDiffuser with Inverse Dynamics Models of Different Horizons.** We present the success rates of CompDiffuser with 6 different inverse dynamics model horizons. In both environments, Large and Giant, our method performs consistently across all configurations, showing that the synthesized plans adhere to the transition dynamics and are easy to follow.

### B.3 Sampling Time Comparison: Parallel vs. Autoregressive

In Table 10, we compare the diffusion denoising sampling time of three methods: (1) monolithic model method, Decision Diffuser (DD), where we directly sample a long trajectory with the same horizon as the proposed compositional sampling method; (2) parallel compositional sampling, as shown in the left of Figure 3, where we denoise all trajectories  $\tau_{1:K}$  in one batch; (3) autoregressive compositional sampling, as shown in the right of Figure 3, where we sequentially denoise each  $\tau_k$ .

We observe that both parallel and AR sampling methods require more sampling time than DD probably due to (1) the simpler and smaller denoiser network of DD; (2) time for condition